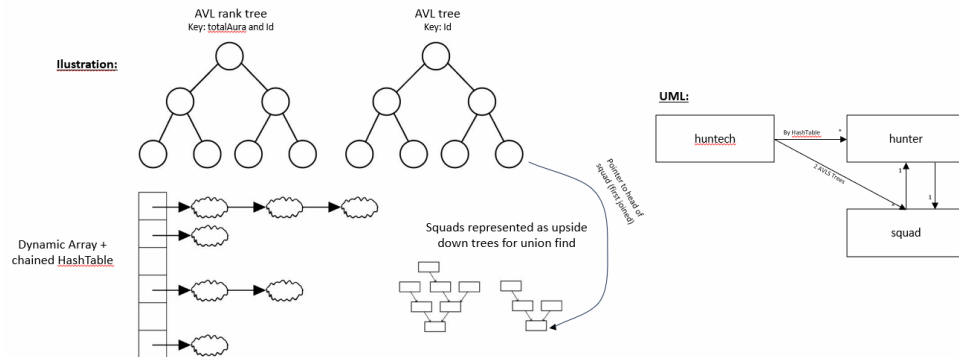


תרגיל רטוב 2 - חלק יבש

תיאור המבנה:

המערכת מבוססת על שילוב של עצי AVL לניהול הקבוצות יחד עם טבלת ערבול (Hash Map) ומימוש של אלגוריתם Union-Find לניהול ההאנטרים. להלן פירוט המבנים בהם השתמשנו:

1. עץ דרגות AVL: עץ המסדר את הקבוצות לפי מפתח משולב של totalAura יחד עם Id על מנת לשלוף הקבוצה ה-i בחוזקה ביעילות.
2. עץ דרגות AVL: עץ נוסף לשמירת הקבוצות מסודר לפי Id לטובת חיפוש הוספה ומחיקה של קבוצה לפי Id (ללא שימוש בדרגות).
3. טבלת ערבול + מערך דינאמי: לטובת גישה מהירה לכל האנטר לפי ה-Id שלו ומחזיקה מצביעים לצמתים בעצים הפוכים של ה-union find.
4. מבנה union_find: מבנה המייצג את קבוצות ההאנטרים כעצים הפוכים, המבנה כולל:
 - א. כיוון מסלולים ליעול חיפוש.
 - ב. שימוש בערכי delta בין האנטר לאבא ישיר שלו כך שסכום המסלול מהאנטר לשורש ייתן את הערך האמיתי.
 - ג. קישור של האנטר לקבוצה בדגש על המנהיג (הראשון שנכנס) לטובת בדיקה על האנטר מסויים אם ח'.



סיבוכיות מקום:

סיבוכיות המקום של המבנה הינה $O(n + k)$ נפרט:

- א. האנטרים: כל האנטר הינו בעל כמות משתנים קבועה ולכן סיבוכיות המקום הינה $O(n)$.
- ב. טבלת הערבול: טבלה דינאמית לינארית לכמות ההאנטרים הקיימים ולכן סיבוכיות המקום הינה $O(n)$.
- ג. עצי הקבוצות: כל קבוצה בעלת כמות משתנים קבועה ולכן כיוון שיש 2 עצים סיבוכיות המקום הינה $O(k)$.

ולכן סיבוכיות המקום הכוללת הינה $O(n) + O(n) + O(k) = O(n + k)$

פונקציות:

לכל פונקציה נתחיל מבדיקת תקינות לקלט ($O(1)$) ואם לא מתאים נחזיר ערך בהתאם לנדרש בתרגיל. כעת נתאר את הפונקציות וננתח את סיבוכיות הזמן שלהן:

:huntech

נאתחל מבנה עם שני עצים (כפי שתואר במבנה) ריקים ובנוסף טבלת ערבול ריקה.

סיבוכיות זמן: סיבוכיות הזמן של אתחול עץ ריק ושל טבלת ערבול ריקה הינו $O(1)$ ולכן כיוון שאנחנו מתאחלים מספר קבוע של עצים וטבלת ערבול סיבוכיות הזמן הכוללת הינה $O(1)$.

:~virtual huntech

נשתמש בהורס הדיפולטיבי כיוון שניהול הזיכרון שלנו ממומש באמצעות מצביעים חכמים `weak_ptr` או `share_ptr` מחלקות שמנהלות זיכרון בעצמן ולכן ההורס הדיפולטיבי פשוט יקרא להורסים שלהם.

סיבוכיות זמן: סיבוכיות הזמן הינה $O(n + k)$ כיוון שהריסת 2 עצי ה-AVL המצביעים הינה $O(k)$ וסיבוכיות מחיקת טבלת הערבול הינו $O(n)$ ולכן סך הכל נקבל $O(n) + O(k) = O(n + k)$.

:add squad

ניצור אובייקט חדש מסוג קבוצה ונרצה להכניס את הקבוצה החדשה לתוך המבנה. כפי שתואר, הקבוצות מוחזקות על ידי שני עצי דרגות: עץ אחד מסודר לפי squadId ועץ שני מסודר לפי totalAura. לכן לאחר בדיקות תקינות נבצע הכנסה של הקבוצה לשני העצים בנוסף, במידה והכנסה לעץ השני נכשלת בגלל בעיית זיכרון, נבצע rollback ונמחק את מה שהוכנס לעץ הראשון כדי לשמור על עקביות המבנה.

סיבוכיות זמן:

בדיקות תקינות הקלט מתבצעות ב $O(1)$ חיפוש הקבוצה במערכת על מנת לוודא שאינה קיימת מתבצע בזמן $O(\log(k))$ כפי שנלמד בכיתה, וכן הכנסה לכל קבוצה מתבצעת ב- $O(\log(k))$. ולכן סך הכל נקבל:

$$O(1) + O(2 * \log(k)) + O(\log(k)) \text{ ולכן סיבוכיות הזמן של הפוקנציה אכן תהיה } O(\log(k)).$$

:remove squad

נרצה להסיר קבוצה מהמבנה. לכן תחילה נבצע בדיקות תקינות קלט ונבדוק האם קיימת קבוצה עם מזהה squadId במערכת. במידה והקבוצה אינה קיימת נחזיר FAILURE. אם הקבוצה קיימת ואינה ריקה, מאחר ואנו עדיין נדרשים לשמור מידע מצטבר על הציידים גם לאחר הסרת הקבוצה, ניגש אל את שורש העץ ההפוך של הציידים בקבוצה ונבצע עדכון של הערכים הרלוונטיים: נוציא את הציידים מהקבוצה על ידי איפוס השיוך לקבוצה בשורש, ונגלגל לשורש את מספר הקרבות של הקבוצה באמצעות deltaFights, כך ששאליות עתידיות על הציידים יחזירו ערכים נכונים גם לאחר מחיקת הקבוצה. לאחר מכן, בהתאם למבנה הנתונים שתואר, הקבוצות מוחזקות על ידי שני עצי דרגות — אחד המסודר לפי squadId השני לפי totalAura לכן נבצע הסרה של הקבוצה משני העצים ובכך נסיר את הקבוצה מהמערכת.

סיבוכיות זמן:

בדיקות תקינות הקלט מתבצעות ב $O(1)$ חיפוש הקבוצה במערכת על מנת לוודא שקיימת מתבצע בזמן $O(\log(k))$ כפי שנלמד בכיתה, גישה לשורש של העץ ההפוך ושינוי הערכים בו מתבצעת ב $O(1)$ וכן הסרת הקבוצות מתבצעת ב $O(2(\log(k)))$. ולכן סך הכל נקבל:

$$O(1) + O(1) + O(\log(k)) + O(\log(k)) \text{ ולכן סיבוכיות הזמן של הפוקנציה אכן תהיה } O(\log(k)).$$

:add hunter

ניצור אובייקט האנטר חדש ונשייך אותו לקבוצה שאליה אמור להיכנס, אם הקבוצה הייתה ריקה נאתחל את ההאנטר להיות המנהיג ואם לא פשוט נעשה unite כפי שנלמד בכיתה לקבוצה. בנוסף לפני הכנסתו לקבוצה נמחק את הקבוצה מעץ הדרגות שמסודר לפי ה-totalAura id-וה ואז נכניס מחדש אחרי צירופו לקבוצה כיוון שמשפיע על ה-totalAura של הקבוצה.

סיבוכיות זמן: סיבוכיות הזמן של יצירת האנטר הינה $O(1)$ וסיבוכיות הזמן המשוערכת של הכנסתו לטבלת הערבול בממוצע על הקלט הינה $O(1)$, סיבוכיות הזמן של הסרה והוספה חזרה לעץ הינה $O(\log k)$ ובנוסף ביצוע unite מתקיים ב $O(1)$ כיוון שנפנה אליה עם שורש העץ ההפוך ולכן החיפוש של השורש בתוכה יהיה ב- $O(1)$ במקום $O(\log * n)$ ולכן סך הכל סיבוכיות הזמן המשוערכת של הפונקציה בממוצע על הקלט הינה:

$$T(n) = O(1) + O(\log k) = O(\log k)$$

:squad duel

ראשית נבדוק אם הארגומנטים תקינים אם כן נגדיל לשני הצוותים את מספר הקרבות באחד, נחשב את הילת הקרב האפקטיבית של כל צוות ואז נבצע בדיקות איזה צוות מנצח ותחת איזה תנאי ונגדיל בהתאם את הניסיון של הצוות (או של שניהם במקרה של תיקו) ונחזיר ערך מתאים.

סיבוכיות זמן: סיבוכיות הזמן הינה $O(\log k)$. זאת כיוון שחיפוש שני הצוותים הינו $O(\log k)$, חישוב הילת הקרב האפקטיבית של כל צוות הינה ב- $O(1)$ כיוון שמחושבת בעזרת שדות של הצוות ולבסוף נבצע לכל היותר 4 השוואות לבדיקה איזה צוות ניצח ונבצע עדכון לצוות המנצח בנוסף להחזרת ערך מתאים כל זה ב- $O(1)$ ולכן סיבוכיות הזמן של הפונקציה הינה

$$T(n) = O(1) + O(\log k) = O(\log k)$$

:get hunter fights number

נאתר את הצומת של הצייד ונבצע קריאה לפונקציה `find`, אשר מחזירה את שורש העץ ההפוך של הצייד ומבצעת `path compression` תוך צבירת ערכי `deltaFights` באופן הנלמד בכיתה כאשר `deltaFights` מייצג את הקרבות שנצברו עבור צייד בקבוצה. לאחר מציאת השורש, נחשב את מספר הקרבות הכולל של הצייד כסכום של מספר הקרבות האישיים של הצייד, הדלתא שנצברה עבורו, ומספר הקרבות של הקבוצה אליה הוא שייך. במידה והצייד אינו השורש, נוסיף גם את ערך `deltaFights` של השורש. באופן זה מתקבל מספר הקרבות הנכון של הצייד גם לאחר פעולות איחוד, קרבות קבוצתיים והסרת קבוצות.

סיבוכיות זמן:

חיפוש הצייד במפת הציידים מתבצע בזמן $O(1)$.
הקריאה לפונקציה `find` מתבצעת ב $O(\log^* n)$ כאשר n הוא מספר הציידים כפי שנלמד בכיתה עבור מבנה Union-Find עם `path compression`.
שאר החישובים והעדכונים מתבצעים בזמן $O(1)$.
לכן סיבוכיות הזמן של הפונקציה `get_hunter_fights_number` היא:

$$O(\log^*(n)) = O(\log^*(n)) + O(1) + O(1)$$

:get squad experience

נבצע חיפוש של הקבוצה בעץ AVL עם המפתח לפי `Id` ואז עם `getter` נחזיר את ערך הנסיון של הקבוצה אשר מתעדכן בכל פעם שנקרא לפונקציה `squad_duel`.

סיבוכיות זמן: סיבוכיות הזמן של חיפוש בעץ AVL הינו $O(\log k)$ במקרה הגרוע כפי שנלמד בכיתה וסיבוכיות הזמן של החזרת ערך הנסיון של הקבוצה הינו $O(1)$ ולכן סך הכל הסיבוכיות הפונקציה במקרה הגרוע הינו $T(n) = O(1) + O(\log k) = O(\log k)$.

:get ith collective squad aura

בעזרת שימוש בעץ הדרגות שלנו המסודר לפי מפתח של totalAura ו-id נשתמש בפונקצייה select(i) של עץ הדרגות כדי שנלמדה בכיתה ונחזיר את האיבר ה-i בגודלו בעץ הדרגות.

סיבוכיות זמן: סיבוכיות הזמן של פעולת select בעץ הדרגות הינה לינארית לגובה העץ ולכן סיבוכיות הזמן של הפונקציה במקרה הגרוע הינו $O(\log k)$.

:get partial nen ability

הפונקציה מאתרת את ההאנטר בטבלת הערבול שלנו ומבצעת פעולת find בעץ ההפוך של הקבוצה של ההאנטר זאת על מנת לכווץ מסלולים ולעדכן את ערכי הערכים היחסיים בין ההאנטרים בקבוצה תוך כדי ולמצוא את השורש (בו נבדוק אם הקבוצה מתה). לבסוף נחשב עבור ההאנטר את כמות ה-nen שלו ועוד ההפרש ב שלו מהאבא הישיר ואם אינו השורש נוסיף גם את ערכי ההפרש nen של השורש שיכול היה להצטבר במהלך איחודי קבוצות.

סיבוכיות זמן: סיבוכיות הזמן המשוערכת בממוצע על הקלט של הפונקציה find הינה $O(\log^* n)$ כפי שנלמד בכיתה בנוסף נבצע עוד מספר קבוע של פעולות כלומר $O(1)$ ולכן סיבוכיות הזמן המשוערכת הכוללת של הפונקציה בממוצע על הקלט הינה $T(n) = O(1) + O(\log^* n) = O(\log^* n)$.

force join:

נרצה לאחד שתי קבוצות כאשר קבוצה אחת כופה על הקבוצה השניה להצטרף אליה.

כדי לשמור על עקביות בעץ הדרגות המסודר לפי totalAura, נסיר את שתי הקבוצות מהעץ בעזרת המפתחות הנוכחיים שלהן.

לאחר מכן נבצע איחוד של שני העצים ההפוכים של הציידים באמצעות unite עם השורשים של כל קבוצה, ה-unite מתבצע לפי rank כאשר rank מייצג את גובה העץ כך שהשורש בעל ה-rank הגבוה יותר נשאר שורש, והעץ בעל ה-rank הנמוך יותר מתחבר אליו. בנוסף, בסיום האיחוד מעדכנים את השורש החדש כך שיחזיק כעת את הקבוצה הכופה וכן נעדכן את הצייד שהקבוצה מחזיקה להיות השורש החדש.

לאחר האיחוד נעדכן את נתוני הקבוצה הכופה כך שישקפו את האיחוד: נוסיף לה את הניסיון של הקבוצה הנכפית, את סכום ה totalAura שלה ואת סכום יכולות ה־Nen. בנוסף נסיר את הקבוצה הנכפית מעץ הקבוצות לפי (squadTree) squadId.

לבסוף נכניס מחדש את הקבוצה הכופה ל-ITree עם מפתח מעודכן לפי הערכים החדשים, ובכך נשמור על סדר נכון גם עבור get_ith_collective_aura_squad.

סיבוכיות זמן:

בדיקות תקינות הקלט הן $O(1)$ חיפוש שתי הקבוצות בעצים מתבצע ב $O(\log(k))$ הסרת הקבוצות מה itree (העץ המסודר לפי totalAura) גם כן בסיבוכיות של $O(\log(k))$ וכן גם הכנסת הקבוצה בחזרה בסוף הפונקציה גם כן לוקחת $O(\log(k))$. פעולת האיחוד בין שתי הקבוצות ב unite מתבצעת בסיבוכיות משוערכת של $O(\log^*(n))$ מכוון שאנחנו מבצעים איחוד ב union find באופן הנלמד בכיתה ומעדכנים את שדות השורש בזמן של $O(1)$. לכן סך הכל נקבל כי סיבוכיות הזמן המשוערכת של הפונקציה היא : $O(\log^*(n) + \log(k))$.